COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - Data Structures and Algorithms	SECTION:	CPE21S2
DATE PERFORMED:	07-17-26	DATE SUBMITTED:	07-21-26
NAME:	Christian Lee C. Bandigan	INSTRUCTOR:	Engr. Jimlord Quejado

ACTIVITY NO. 2 Hands-on Activity 2.1 Arrays, Pointers and Dynamic Memory Allocation

1. PROCEDURE OUTPUT

Screenshot

HOA2.1 > procedure > procedure.cpp > ...

```cpp
#include <iostream>
#include <string.h>

class Student {
    private:
        std::string studentName;
        int studentAge;
    public:
        //constructor
        Student(std::string newName = "John Doe", int newAge = 18){
            studentName = std::move(newName);
            studentAge = newAge;
            std::cout << "Constructor Called." <<std::endl;
        };

        //destructorS
        ~Student(){
            std::cout << "Destructor Called." <<std::endl;
        }

        //Copy Constructor
        Student(const Student &copyStudent){
            std::cout << "Copy constructor Called." <<std::endl;
            studentName = copyStudent.studentName;
            studentAge = copyStudent.studentAge;
        };

        //Copy Assignment Operator
        Student& operator=(const Student& copyStudent){
            std::cout << "Copy Assignment Operator Called." << std::endl;
            if(this != &copyStudent){
                studentName = copyStudent.studentName;
                studentAge = copyStudent.studentAge;


            }

            return *this;
        };
        //Display Attribute
        void printDetails(){
            std::cout << this->studentName << " " << this->studentAge << std::endl;
        };

};
```

```
45  int main(){
46      Student student1("Christian",28);
47      student1.printDetails();
48      Student student2(student1);
49      student1.printDetails();
50      Student student3;
51      student3 = student2;
52      student1.printDetails();
53
54      return 0;
55  }
```

```
PS C:\Users\TIPQC\Documents\CPE010 - Bandigan\HOA2\procedure> cd "c:\Users\TIPQC\Documents\
Constructor Called.
Christian 28
Copy constructor Called.
Christian 28
Constructor Called.
Copy Assignment Operator Called.
Christian 28
Destructor Called.
Destructor Called.
Destructor Called.
PS C:\Users\TIPQC\Documents\CPE010 - Bandigan\HOA2\procedure>
```

Observation	In this activity it shows how to properly create a class with the big three. The big three are of course the destructor, copy constructor, and copy assignment operator. The destructor is done with the tilde together with the class name. It is called after each run and it just deletes the objects of the class. The copy constructor is used to create a variable as a copy of an existing object. It is done by creating an object and inside the parameter is an existing object. The copy assignment operator is used to copy an existing object to an existing object. It is done by assigning an object to another object. As you can see in the output the constructor, copy constructor, and copy assignment operator are all called once so the objects all have the same name and age. The destructor is called three times because of the three objects.

Table 2-1. Initial Driver Program

Screenshot

```
33  int main(){
34      const size_t j = 5;
35
36      Student studentList[j] = {};
37      std::string namesList[j] = {"Carly", "Freddy", "Sam", "Zack", "Cody"};
38      int ageList[j] = {15,16,18,19,16};
39
40      return 0;
41  }
```

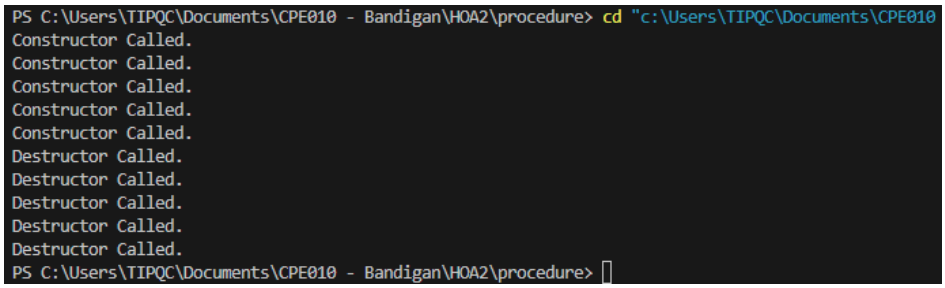
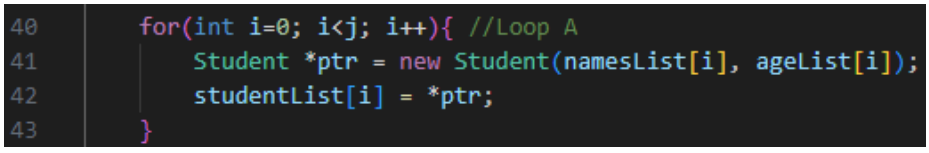
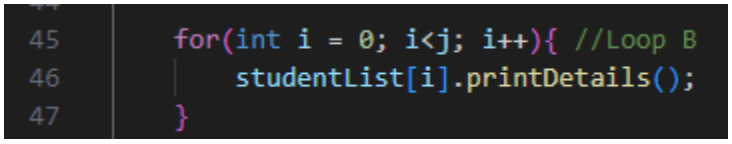
	
Observation	In this modified driver program it shows how static allocation works by making an array of objects, student names, and student ages. As you can see in the output the constructor and destructor are called five times because of the five objects.

Table 2-2. Modified Driver Program with Student Lists

Loop A	
Observation	This for loop is used to insert the data of names and age into the array of objects. It is done by dynamically allocating memory to a pointer each iteration and assigning it to the array.
Loop B	
Observation	This for loop is used to print the details of each object in the array.

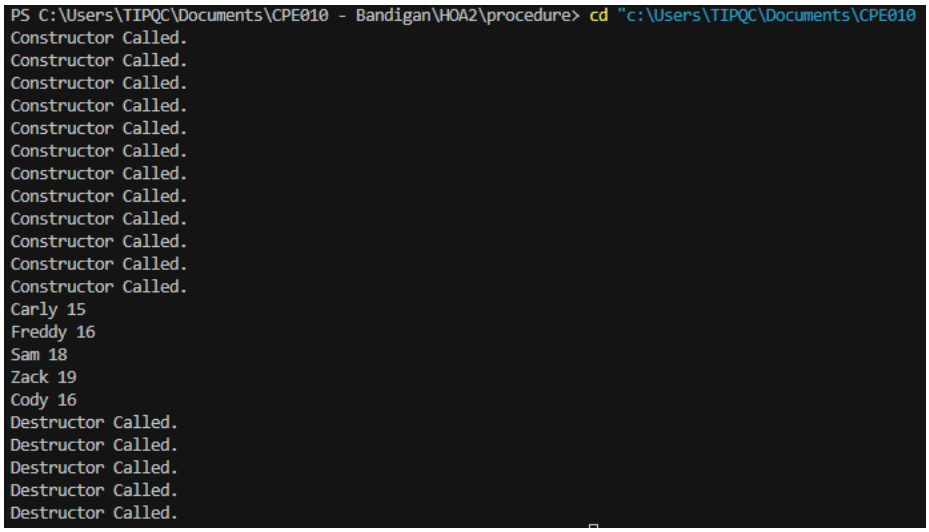
Output	
Observation	The functions worked correctly.

Table 2-3. Final Driver Program

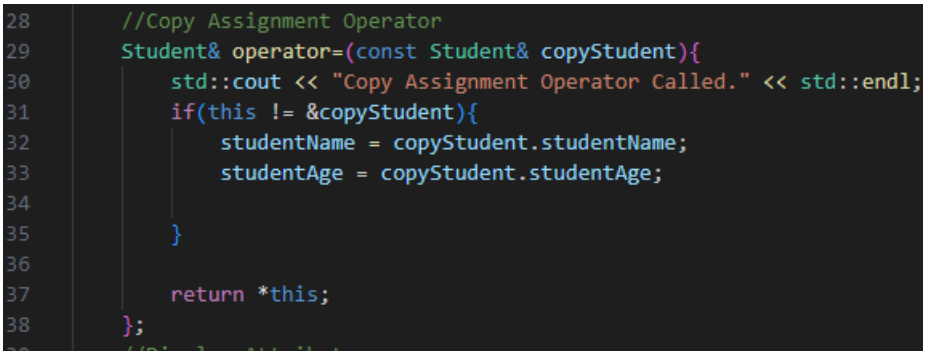
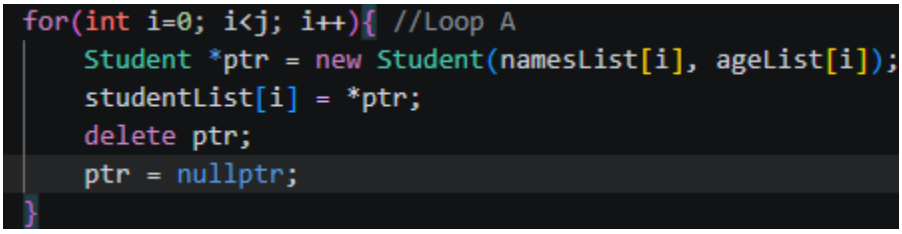
Modifications	
Observations	This modification adds the copy assignment operator that is called when you want to copy an object to an existing object.
Modifications	
Observations	This is a modification in loop A. I added delete ptr to avoid memory leak and set it to nullptr to avoid dangling pointers. This is how to properly allocate dynamic memory using AUDNU! (Allocate, Use, Delete, null)

Table 2-4. Modifications/Corrections Necessary

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

HOA2.1 > supplementary > ILOC.cpp > ...

```cpp
#include <iostream>
#include <string.h>

class Fruit{
    private:
        std::string name;
        float price;
        int quantity;
    public:
        //constructor
        Fruit(std::string newName="apple", float newPrice = 20, int newQuan=1);
        //destructor
        ~Fruit();
        //copy constructor
        Fruit(const Fruit &copyFruit);
        //copy assignment operator
        Fruit& operator=(const Fruit& copyFruit);
        void printDetails();
        float getTotal();
};
class Vegetable{
    private:
        std::string name;
        float price;
        int quantity;
    public:
        //constructor
        Vegetable(std::string newName = "eggplant", float newPrice=20, int newQuan=1);
        //destructor
        ~Vegetable();
        //copy constructor
        Vegetable(const Vegetable &copyVegie);
        //copy assignment operator
        Vegetable& operator=(const Vegetable& copyVegie);
        void printDetails();
        float getTotal();
};

float totalSum(Fruit fruits[], Vegetable veggies[], int size);
```

Figure 1. Class and Function Declaration

```cpp
int main(){
    const size_t j = 2;
    int veg_size = 2;

    Fruit* grocery_fruit = new Fruit[j]{Fruit("Apple", 10,7),Fruit("Banana",10,8)};
    Vegetable* grocery_vegie = new Vegetable[j]{Vegetable("Broccoli",60,12),Vegetable("Lettuce",50,10)};

    std::cout << "----Jenna's Grocery List----" << std::endl;
    for(int i = 0;i<2;i++){
        grocery_fruit[i].printDetails();
    }
    for(int i = 0;i<2;i++){
        grocery_vegie[i].printDetails();
    }

    std::cout << "--------------------" << std::endl;
    std::cout << "Total Price: PHP"<<totalSum(grocery_fruit,grocery_vegie,j)<<std::endl;

    Vegetable* temp = new Vegetable[veg_size - 1];

    temp[0] = grocery_vegie[0];

    delete[] grocery_vegie;

    grocery_vegie = temp;
    veg_size--;

    delete[] grocery_fruit;
    delete[] grocery_vegie;

    return 0;
}
```

Figure 2. Main

```
74  Fruit::Fruit(std::string newName, float newPrice, int newQuan){
75      //std::cout << "constructor" <<std::endl;
76      name = newName;
77      price = newPrice;
78      quantity = newQuan;
79  }
80  Fruit::~Fruit(){
81      //std::cout << "destructor" << std::endl;
82  }
83  Fruit::Fruit(const Fruit &copyFruit){
84      //std::cout << "copy constructor" << std::endl;
85      name = copyFruit.name;
86      price = copyFruit.price;
87      quantity = copyFruit.quantity;
88  }
89  Fruit& Fruit::operator=(const Fruit& copyFruit){
90      if (this != &copyFruit){
91          //std::cout << "copy assignment operator" << std::endl;
92          name = copyFruit.name;
93          price = copyFruit.price;
94          quantity = copyFruit.quantity;
95      }
96      return *this;
97  }
98  void Fruit::printDetails(){
99      std::cout << this->name << " PHP" << this->price <<" x"<< this->quantity << std::e
100 }
101
102 float Fruit::getTotal(){
103     return price *quantity;
104 }
```

```cpp
Vegetable::Vegetable(std::string newName, float newPrice, int newQuan){
    //std::cout << "constructor" <<std::endl;
    name = newName;
    price = newPrice;
    quantity = newQuan;
}
Vegetable::~Vegetable(){
    //std::cout << "destructor" << std::endl;
}
Vegetable::Vegetable(const Vegetable &copyVegie){
    //std::cout << "copy constructor" << std::endl;
    name = copyVegie.name;
    price = copyVegie.price;
    quantity = copyVegie.quantity;
}
Vegetable& Vegetable::operator=(const Vegetable& copyVegie){
    if (this != &copyVegie){
        //std::cout << "copy assignment operator" << std::endl;
        name = copyVegie.name;
        price = copyVegie.price;
        quantity = copyVegie.quantity;
    }
    return *this;
}
void Vegetable::printDetails(){
    std::cout << this->name << " PHP" << this->price <<" x"<< this->quantity << std::endl;
}

float Vegetable::getTotal(){
    return price *quantity;
}

float totalSum(Fruit fruits[], Vegetable veggies[], int size){
    float total = 0;

    for(int i = 0; i <size; i++){
        total += fruits[i].getTotal();
        total += veggies[i].getTotal();
    }
    return total;
}
```

Figure 3.

```
----Jenna's Grocery List----
Apple PHP10 x7
Banana PHP10 x8
Broccoli PHP60 x12
Lettuce PHP50 x10
--------------------
Total Price: PHP1370
PS C:\Users\User\OneDrive\Documents\CPE010_DSA\HOA2.1\supplementary>
```

Figure 4.

Analysis

I first created two classes, Fruit and Vegetable, each with constructors, destructors, copy constructors, copy assignment operators, and member functions for printing details and calculating the total cost. In the main() function, dynamic arrays of fruits and vegetables are created using new[] allowing dynamic memory allocation. After that I printed the grocery list using a for loop and the print method and it also computes the total price of all items. To remove the lettuce from the vegetable list I created a new smaller dynamic array, Broccoli is copied into it, the old array is deallocated using delete[], and the pointer is updated to the new array. Finally, all dynamically allocated memory is released using delete[] to prevent memory leaks.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.